For each of them, we compute the distance between target (point `Z` in the figure) and the spheres' centroids, as shown in listing 10.4, describing the pseudocode implementation of method `SsNode::intersectsPoint`; since for both the spheres the computed (Euclidean) distance is smaller than their radii, this means that either (or both) could contain our target point, and therefore we need to traverse both `S1` and `S2` branches.

This is also apparent in figure 10.10, where clearly point `Z` lies in the intersection of spheres `S1` and `S2`.

The next couple of steps, in figures 10.10 (bottom half) and 10.11 execute the same lines of code, cycling through node's children until we get to a leaf. It's worth noting that this implementation will perform a depth-first traversal of the node: it sequentially will follow paths down to leaves, getting to leaves as fast as possible, back-tracking when needed. For the sake of space, figures show these paths as they were traversed in parallel: which is totally possible, with some modifications to the code (that would, however, be quite dependent on the programming language of an actual implementation, so we will stick with the simpler – and less resource-intensive – sequential version).

The method will sometime traverse branches where none of the children might contain target: that's the case, for instance, of the node containing `S3` and `S4`. The execution will just end up at line #9 of listing 10.3, returning null and back-tracking to the caller: it had initially traversed branch `S1`, now the `for-each` loop at line #5 will just move on to branch `S2`.